Overcoming Early-Life Failure and Aging for Robust Systems

Yanjing Li, Young Moon Kim, and Evelyn Mintarno
Stanford University

Subhasish Mitra
Stanford University

Donald S. Gardner
Intel Labs

Editor's note:
The prospect of system failure has increased because of device- and chip-level effects in the late CMOS era, such as early-life failure and NBTI. In this article, the authors present novel system-level architecture and design innovations to cope with these lifetime reliability challenges.

—Pradip Bose, IBM Research

Two prevailing notions influence the design of most computer systems today (except high-end mainframes and safety-critical systems):

- The underlying hardware will always produce correct outputs during its useful lifetime.
- Hardware-induced error rates are typically much lower compared to software or operator errors.

At nanometer-scale geometries, several hardware failure mechanisms, which were largely benign in the past, are becoming visible at the system level. Moreover, recent studies indicate that, depending on the application, hardware failures can be significant contributors to overall system failure rates.[1,2] Hardware failures can have severe consequences: for example, on 6 August 1999, eBay's servers crashed and resulted in more than eight hours of downtime; the cause was attributed to hardware failure (see http://findarticles.com/p/articles/mi_m0CGN/is_3720/ai_55394676). Other high-profile Web sites, such as Amazon and PayPal, experienced similar outages because of hardware failures (see http://news.cnet.com/Amazon-fixes-daylong-hardware-glitch/2100-1017_3-273042.html and http://www.v3.co.uk/v3/news/2247234/paypal-outage-hits-ebay). Such failures can have enormous financial consequences. Robust computing systems must continue to meet user expectations despite rising levels of disturbances in the underlying hardware.

Design of robust systems ensuring required hardware reliability, although nontrivial, is achievable but at high costs. Concurrent error detection during system operation is an extremely important aspect of such systems. Some errors are easy to detect—for example, accesses to illegal memory locations, divide by 0, or program hangs. Life would be simple if all errors manifested in these ways. Subtle errors, however, can happen—errors producing incorrect results or incorrect program control flow, for example—that might not be detected by such simple checks. Such errors, if not detected, can have severe consequences, ranging from data loss to financial and productivity losses or even loss of human lives.

Several error detection techniques exist, but they are generally expensive. The most significant challenge is to achieve acceptable levels of robustness at minimum system-level costs (power, performance, area, and design complexity). Furthermore, many error detection techniques often assume a single faulty component and further assume that failures are independent. Such assumptions, however, might not be adequate in the future.

0740-7475/09/$26.00 © 2009 IEEE Copublished by the IEEE CS and the IEEE CASS

Fortunately, thanks to new cost constraints in future technologies and emerging killer applications, several new opportunities for cost-effective solutions have arisen, such as

- possible availability of an abundance of transistors, (although long wires and power continue to pose major challenges);
- the availability of high-speed and high-density nonvolatile storage;
- wide proliferation of multicore architectures; and
- some degree of resilience in several emerging workloads, such as recognition, mining, and synthesis (RMS).

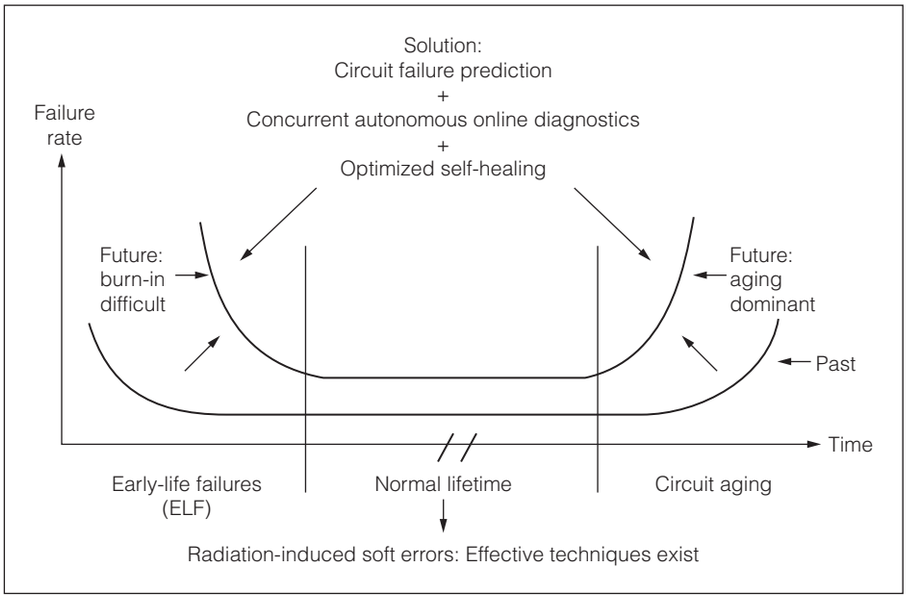

Figure 1. Reliability challenges for robust system design.

Hardware reliability challenges arise from three major sources: early-life failures (also called infant mortality), radiation-induced soft errors, and circuit aging, as summarized in Figure 1. The impact of these challenges might well become more severe for future systems.[3] (Although design errors and electrical bugs in complex systems constitute a major challenge and must also be addressed for future robust systems, they are beyond the scope of this article.)

Several techniques, such as *Built-In Soft-Error Resilience (BISER)*, can be effectively used for correcting radiation-induced transient (soft) errors.[4] In this article, however, we focus on *early-life failures (ELF)* and *circuit aging*. Table 1 summarizes the associated failure characteristics and challenges for early-life

Table 1. Characteristics and challenges of early-life failures (ELFs) and circuit aging.

Reliability challenges	Causes	Existing solutions	Challenges
Early-life failure (ELF)	Manufacturing defects:[5] ■ Gate-oxide defects ■ Back-end defects (ELF from interconnect voids are rare in 90 nm)	■ Burn-in ■ High-voltage stress test ■ I_{DDQ} test ■ Very low-voltage (VLV) and min V_{DD} test ■ Outlier analysis	■ Burn-in increasingly difficult:[3] high power dissipation and cost, potentially reduced effectiveness ■ Burn-in alternatives face major challenges because of excessive circuit leakage, reduced voltage margins, and increased process variations
Circuit aging	Major sources: PMOS negative-bias temperature instability (NBTI) and NMOS positive-bias temperature instability (PBTI)	Worst-case speed and voltage margins (guardbands)	One-time worst-case guardbands pessimistic and may be too expensive for future systems[6]

failures and circuit aging. We present an overview of techniques for designing robust systems that can overcome early-life failures and aging:

- Circuit failure prediction for early failure indication
- Concurrent, autonomous, online diagnostics
- Optimized self-healing

These techniques utilize specific characteristics of reliability mechanisms without incurring the high costs of traditional concurrent error detection.

Existing concurrent error detection techniques

Several concurrent error detection techniques exist. On-chip memories routinely use error correcting codes (ECC), along with scrubbing and sparing. For errors in logic, existing error detection techniques are generally expensive. Duplication and triple modular redundancy (TMR) have been used in commercial systems, such as IBM's z990 and HP's Non-Stop Advanced Architecture. These techniques incur significant power, performance, and area penalties, and they introduce additional system design complexities.

Coding techniques can be used for error detection in logic circuits.[7] Classical parity codes for logic, known as *parity prediction*, constrain the amount of logic sharing that can be allowed among the circuits producing outputs being checked. These constraints result in significant power, performance, and area penalties for arbitrary logic functions. Relaxing logic-sharing constraints can reduce the associated penalties of parity codes, but at the price of coverage.

Time redundancy techniques for error detection, using a variety of software and hardware techniques (and combinations thereof), reduce some of the area and power overheads of duplication but introduce additional performance penalties (between 20% to more than 100%, depending on the implementation).

Error detection costs can be reduced in two ways:

- by detecting a subclass of errors with special properties, such as low-cost delay fault detection; or
- by using special application properties, such as inexpensive checkers for matrix operations or assertions.

Furthermore, existing error recovery techniques such as various checkpoint and rollback schemes, used in conjunction with error detection techniques, can be complex and difficult to validate.

Circuit failure prediction

Circuit failure prediction provides an early indication of the occurrence of a circuit failure before errors actually appear in system data and states.[8] This is in contrast to concurrent error detection, in which a failure is detected after errors appear in system data and states.

Not all circuit failures can be predicted—for example, radiation-induced soft errors. Circuit failure prediction is ideal for circuit aging and gate-oxide early-life failures because of the gradual degradation associated with these mechanisms. Such gradual degradation results in delay shifts (not necessarily delay faults) over time, which can serve as signatures for early indicators of circuit failures. Upon circuit failure prediction, appropriate self-healing actions must be invoked.

One way to implement circuit failure prediction is to collect information about the gradual evolution of various system parameters over time, and to analyze the collected data to provide early indications of failure. System parameters that we focus on in this article involve timing characteristics of logic signals that provide delay shift information. Additional system usage information such as thermal and voltage profiles, and power states (e.g., on/off states), might also be useful. The information can be collected concurrently during normal system operation using special circuits called *failure prediction sensors* (which differ from traditional process monitors) and/or using periodic online diagnostics. Information required for circuit failure prediction needs to be collected only periodically for short periods of time, not continuously, while a system is in normal operation. Hence, the system power impact of circuit failure prediction can be small.

The idea of circuit failure prediction is orthogonal to concurrent error detection and hard failure detection using periodic online diagnostics. Table 2 compares and contrasts these approaches. However, significant synergies and trade-offs exist between these approaches, which can be utilized for optimized robust system design.

Circuit failure prediction for aging

Here, we illustrate circuit failure prediction for negative-bias temperature instability (NBTI)-induced

Table 2. Circuit failure prediction vs. concurrent error detection and hard failure detection through periodic online diagnostics.

Approach*	Timing of detection	Corrupted data or states exist?	Cost	Applicability	Recovery mechanism
Circuit failure prediction	Before errors appear	No	Low cost (no concurrent checking)	Applicable for certain reliability mechanisms	No error recovery (self-healing required)
Concurrent error detection	After errors appear	Yes, but errors detected	High cost due to concurrent checking	General applicability	Error recovery (self-healing required)
Hard failure detection through periodic online diagnostics	After errors possibly appear	Possibly	Can be expensive for very frequent tests	Broader applicability than prediction, narrower than concurrent error detection	Error recovery (self-healing required)

* Circuit failure prediction, concurrent error detection, and hard failure detection through periodic online diagnostics can be effectively combined.

PMOS transistor aging, a dominant aging mechanism. It is also applicable, however, for other aging mechanisms, such as positive-bias temperature instability (PBTI). The PMOS threshold voltage degrades as a result of NBTI, resulting in increased circuit delay. The amount of threshold voltage degradation of a PMOS transistor depends on the amount of time elapsed, temperature and voltage profiles, and the workload that determines the amount of time the PMOS transistor is on. Different PMOS transistors in a given circuit can degrade at different rates; the overall (cumulative) effect is manifested as shifts in delays of various circuit paths. Designers traditionally incorporate one-time worst-case voltage and frequency guardbands at the beginning of a system's lifetime to prevent possible delay faults due to aging over the entire lifetime (e.g., 7 to 10 years for enterprise systems) even under worst-usage conditions. By examining delay shifts, circuit failure prediction can be effectively used to minimize such aging-related guardbands (see Figure 2).

Instead of one-time worst-case guardbands, the system could start off with a far smaller guardband that guarantees correct system functionality for a short period of time (compared to overall system lifetime)—for example, one to five days, under worst aging. The designer's choice of this initial short period of time depends on several factors, such as the specific aging mechanism, circuit failure prediction implementation, and system support for optimized self-healing. During this time, we use circuit failure prediction to estimate the system's actual aging by collecting lots of data about relative aging of various circuit paths. This data can be collected in two ways:

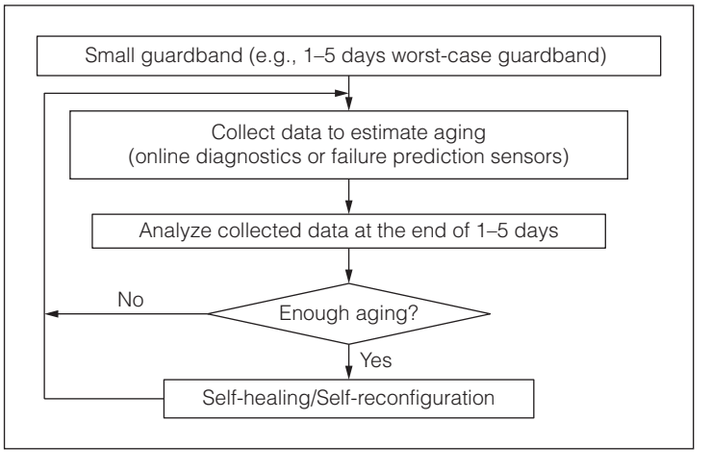

Figure 2. Circuit failure prediction for aging.

- Concurrently during system operation using special flip-flops with *built-in aging sensors*. Care must be taken to ensure that the aging sensors themselves are aging resistant.[8]
- Periodically scheduling special online system diagnostics. Accurate aging estimation requires online diagnostics to thoroughly exercise a circuit using special test patterns.[9] Special test clock control techniques may be adopted for this purpose.

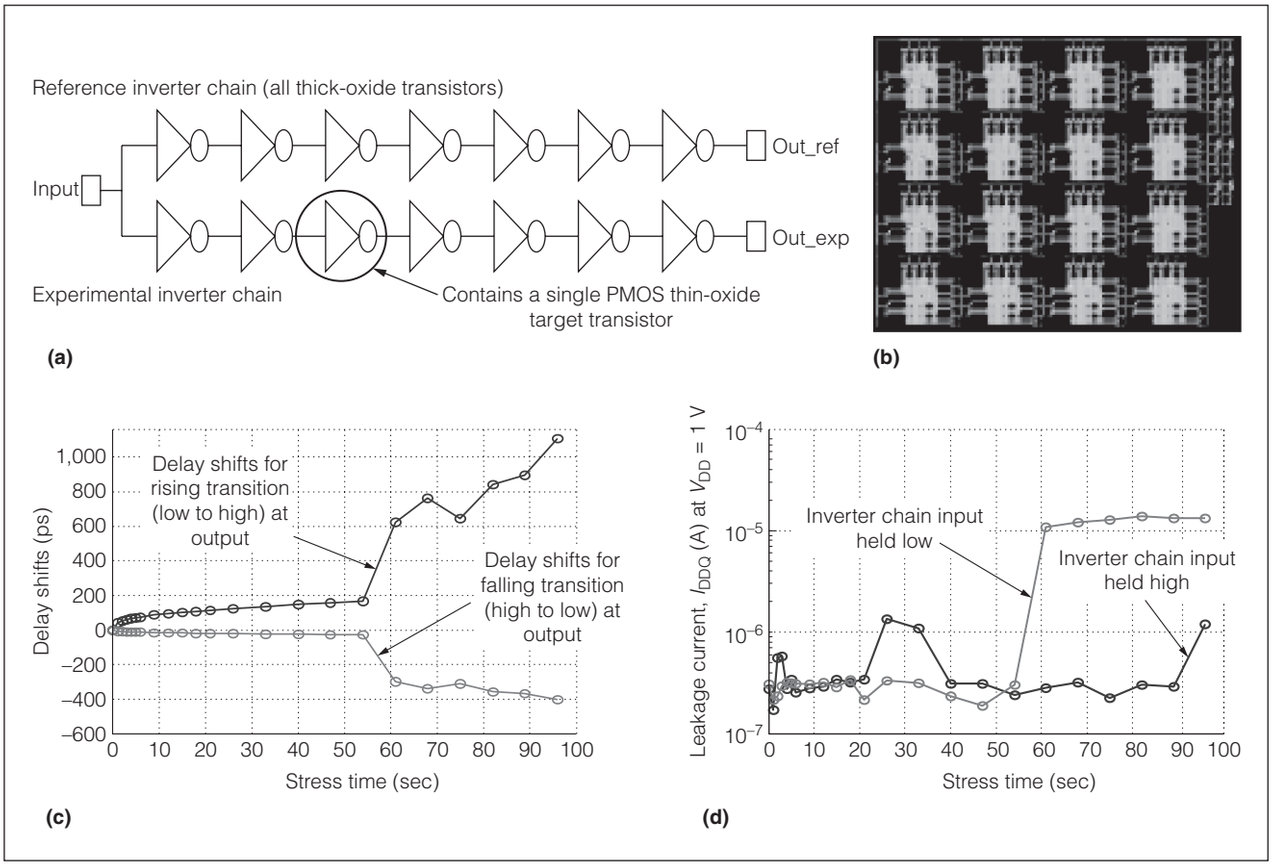

Figure 3. Schematic of inverter chain pair under test (a), test chip layout (b), 3.8-V DC stress results while the inverter chain input is held low, demonstrating gradual delay shifts (c), and leakage current (I_{DDQ}) results (d).

Instead of relying on process monitors such as ring oscillators and temperature sensors, this approach extracts actual aging information directly from the circuits. On the basis of the actual aging, optimized self-healing techniques are invoked to properly adjust the associated guardbands to prevent errors.

Circuit failure prediction for gate-oxide ELFs

We analyzed gate-oxide ELF behaviors through stress experiments using 90-nm test chips (with SiO_xN_y as the gate dielectric)[10] and obtained a new gate-oxide ELF signature (which is distinct from signatures for aging mechanisms such as NBTI, PBTI, and hot electrons):

- At the transistor level, a gate-oxide ELF candidate transistor produces gradually degraded drive currents over time before it completely loses its transistor characteristics;
- At the circuit level, the presence of a gate-oxide ELF-candidate transistor in a digital logic circuit results in delay shifts (along circuit paths containing the ELF candidate transistor) over time before functional failures appear. Note that, delay shifts are different from delay faults.

Figure 3 illustrates some results obtained from stress experiments using 90-nm test chips. Figure 3a shows an inverter chain pair on the test chip; Figure 3b shows the test chip layout. One inverter chain of the pair, the *experimental inverter chain*, contains a single target PMOS thin-oxide transistor in its third stage, whereas the rest of the transistors contain thick-oxide gates. The other inverter chain, referred to as the *reference inverter chain*, has the same design as the experimental inverter chain except that it is made entirely out of thick-oxide transistors. We stressed this inverter chain pair at a high DC stress voltage of 3.8 V to emulate gate-oxide ELF behavior in the thin-oxide transistor of the experimental inverter chain. This emulation relies on the fact that a defective gate-oxide contains more broken Si-Si bonds

compared to a defect-free gate-oxide. Figure 3c shows gradual delay shifts that are induced by a single gate-oxide ELF-candidate transistor in a seven-stage inverter chain in 90-nm technology.[10] Here, *delay shift* is defined as follows: $DS(t) = (D_E(t) - D_R(t)) - (D_E(0) - D_R(0))$ where $DS(t)$ is the delay shift at time t, $D_E(t)$ is the delay of the experimental chain at time t, $D_R(t)$ is the delay of the reference chain at time t, and $D_E(0)$ and $D_R(0)$ are the delays of the experimental and reference chains at time 0.

A positive (or negative) value of delay shift indicates that the experimental inverter chain got slower (or faster) over time. In Figure 3c, the positive delay shifts correspond to rising output transitions, whereas the negative ones correspond to falling output transitions. Such behaviors can be explained by gradually degraded drive currents produced by the thin-oxide PMOS transistor (under stress) in the third stage of the experimental inverter chain. The specific delay shift values are very large because the channel length L of all transistors (both thin-oxide and thick-oxide) was 400 nm as a result of the test chip experiment constraints (instead of 100 nm, which is typical for 90-nm technology). With a 400-nm channel length, the FO4 (fanout of 4) delay of the thin-oxide transistor is considerably larger than the 100-nm channel length. In reality, the delay shifts due to the gate-oxide ELF-candidate transistor are expected to be smaller (e.g., 20 ps).

Although Figure 3c shows a specific example of a PMOS gate-oxide ELF candidate, we confirmed our results for NMOS gate-oxide ELF candidates as well. Moreover, our gate-oxide ELF signature is distinct from aging mechanisms such as NBTI, PBTI, and hot electrons, because of the following reasons:

- We reproduced our results for large arrays of 90-nm NMOS transistors, and also for inverter chains with thin-oxide NMOS transistors. It is well-known that PBTI effects are minimal for NMOS at 90 nm.
- Hot-carrier effects aren't expected to be dominant, because all transistors were in static states (i.e., there was no switching) during voltage stress.
- As Figure 3d shows, I_{DDQ} undergoes a considerable increase (after 61 seconds of stress) when the inverter chain input is held low. We measured the I_{DDQ} in a quiescent state for both high and low logic values at the inverter chain pair's input. However, almost no I_{DDQ} increase occurs when the inverter chain pair's input is held high. Such difference indicates the existence of gate-source resistive paths in the corresponding thin-oxide transistor. Aging effects such as NBTI are not associated with increased I_{DDQ}.

These results are significant because they demonstrate that delay shifts exist and can be reliable indicators for gate-oxide ELFs. This signature can be used to overcome ELF challenges in a few ways:

- New gate-oxide ELF-screening techniques as alternatives to traditional burn-in can be developed by detecting delay shifts (rather than delay faults) inside circuits after applying "short" stresses.
- Circuit failure prediction, together with online diagnostics, can be used to collect comprehensive delay shift information within a chip. Once gate-oxide ELF-candidate transistors are foreseen, appropriate self-healing actions can be invoked before an ELF transistor causes system failures.

Several important questions arise, however, in the context of such ELF identification techniques. First, how often should circuit failure prediction be performed for ELFs? If delay shifts associated with gate-oxide ELFs are less gradual than those associated with aging, online diagnostics and analysis might need to be carried out more frequently than those for aging. An adaptive scheme can collect information about delay shifts and other system parameters very frequently to start with, and then adjust the frequency of data collection on the basis of past history.

Second, what test patterns must be applied and how? One way of detecting relative delay shifts is to use delay test patterns—for example, test patterns targeting transition faults or small delay faults—which are used for production testing. In addition, we might need to record the maximum clock frequency for which the same test pattern passes the test at two different time instants (t_1 and t_2) or variations thereof. Such an approach might require faster-than-at-speed testing. For such techniques to identify ELF candidates during system operation, proper care must be taken to minimize the effects of changes in temperature and voltage profiles. Unlike traditional faster-than-rated-speed delay test techniques, however, delay shift detection is expected to be less sensitive to process variations, noise, and overkill (yield loss) problems. This is because delay shift detection relies

Table 3. Comparison between CASP and pseudorandom logic BIST.

Feature	CASP	Pseudorandom logic BIST
Coverage	High	Generally low
Area cost	Low	High for higher coverage
Test data storage	Moderate but not expensive	Low
Test time impact	Short for system; very little with spare or idle cores	Targets test floor during production tests
Design effort	Low	High
Flexibility	High	Low
Upgradability	Yes	Difficult

on relative shifts over time rather than absolute measurements.

Although the data we've presented is from 90-nm test chips, further experimentation is required to confirm such gate-dielectric ELF behaviors for 45-nm high-*k* metal gate transistors. Moreover, experimental data is necessary to understand the effects of back-end defects—the other major ELF source.

CASP: concurrent autonomous chip self-test and diagnostics using stored test patterns

Effective circuit failure prediction requires efficient and thorough online diagnostics. Such a feature allows a system to test itself concurrently during normal operation without any downtime visible to an end user. One such technique is *CASP* (Concurrent Autonomous chip self-test and diagnostics using Stored test Patterns[11]). CASP enables effective circuit failure prediction for robust system design. In addition, CASP can be used for hard failure detection and failure diagnosis for systems with built-in self-healing. To detect hard failures using CASP, however, proper failure recovery support such as checkpoint and rollback is necessary.

CASP overview

The basic idea behind CASP is to store thorough test patterns with very high test coverage in off-chip storage such as flash memory, and provide architectural and system-level support for testing one or more cores in a multicore system while the rest of the system continues to operate normally.[11,12] CASP avoids limitations of traditional pseudorandom logic BIST but retains its benefits, as a result of the following features:

- *High test coverage*. We need comprehensive tests for online diagnostics—not only stuck-at tests, but also various delay tests such as transition tests, small-delay fault tests, Cadence Encounter True-Time tests, and path delay tests. With CASP, thorough test patterns, both structural and functional (which may include high-quality production tests), can be stored in nonvolatile storage, such as flash memory. Tests that might not be applied during production for test cost reasons (e.g., special test patterns to assist circuit failure prediction for aging[9]) can also be included. Moreover, test patterns can be changed (e.g., using patches) according to application requirements and failure characteristics even after a system is deployed in the field.
- *No visible system-level downtime*. CASP takes advantage of the proliferation of multicore and many-core systems, and incorporates special system support so that online diagnostics can be performed on one or more cores while other cores continue to run normal operations; therefore, there is no downtime visible to an end user.
- *Minimal system-level performance and power impact*. CASP uses efficient test compression techniques to minimize the overall test data volume and test time. Moreover, higher-level system layers, such as virtual machine monitors or operating systems, can optimize trade-offs among reliability, performance, and power.
- *Low cost*. CASP uses the availability of high-speed, high-density, and low-cost off-chip nonvolatile storage such as flash memory to store high-quality test patterns, and uses DFT features—for example, scan chains, test compression such as X-Compact, and at-speed test support.
- *Minimal design flow impact*. CASP requires three elements: a small test controller that instruments online diagnostics, a small on-chip buffer to store a set of test pattern, and appropriate architectural and system support to isolate a core for concurrent online diagnostics without affecting the operation of other cores.

Table 3 compares and contrasts CASP and logic BIST.

CASP implementation and optimization

Our implementation of CASP in the open-source OpenSparc T1 processor, with eight processor cores supporting 32 threads, demonstrates its practicality and effectiveness.[11] We generated comprehensive test patterns for this design with high test coverage: 99.5% coverage for stuck-at faults, 96% for transition faults, and 93.5% for the True-Time faults, using commercial ATPG tools. The required hardware modification was negligible: $< 0.01\%$ overhead of the total chip area, a 7.5-Kbyte on-chip buffer, and a 6-Mbyte off-chip nonvolatile storage per core with $10\times$ test compression in the OpenSparc T1.[11] The total test time is 300 ms per core with flash memory, assuming $10\times$ test compression. The system-level power impact of CASP is also minimal. We estimated the power consumption using a 1-0-1 transition count for the OpenSparc T1, and the power impact of CASP was similar to that of normal applications.[12]

CASP implemented entirely in hardware, that is, *hardware-only CASP*, without any interactions with or support from high-level system layers (e.g., operating systems or virtual machine monitors), has the following limitations. First, robust handling of I/O requests (including interrupts) can be difficult. During online diagnostics, because the core under test is unable to service I/O requests, these requests must be buffered in queues. Depending on hardware queue sizes, some requests might be dropped, and application-level intervention might be required.

Second, online diagnostics can introduce visible application performance impact because of the following reasons:

- When CASP uses scan chains for high test coverage, applications cannot be executed simultaneously on the same piece of hardware.
- Because online diagnostics are invoked to ensure overall system reliability, they cannot be optional.
- Depending on the targeted reliability mechanism, online diagnostics might be executed frequently (e.g., failure prediction for ELFs might occur more frequently than aging) or they might last for a long time (e.g., special test patterns to be applied for transistor-aging prediction might require long online diagnostics times).[9] Although it is possible to use idle intervals in the system for test execution, there is no guarantee that critical or interactive tasks will not be initiated during online diagnostics. If tasks are suspended, there can be a significant impact on the application performance, especially for soft real-time applications (e.g., video/audio playbacks), and interactive applications (e.g., text editors or Web browsers).

CASP can be optimized to overcome these limitations—for instance, through *Virtualization-Assisted concurrent autonomous Self-Test (VAST)*,[12] which uses OS migration to move the execution of an OS from one core to another by coordinating the virtual machine monitors running on the source and destination cores. OS migration enables concurrent online diagnostics when additional cores are available, thereby minimizing application performance impact. We demonstrated our VAST idea on an ARM MPCore system,[12] as Figure 4a shows. To guarantee that the core under diagnostics is properly isolated from the rest of the system, special hardware structures, such as those compliant with IEEE Std 1500, are necessary. We showed that the performance impact of VAST on various CPU-bound, memory-bound, and parallel applications is negligible (e.g., $<1\%$) when spare or idle cores are available.[12] Figure 4b shows that the performance overhead of the SPEC 95 benchmark suite is nearly 0% with VAST, compared to 26% with hardware-only CASP.

Even if spare cores are unavailable, the system-level performance impact of CASP can be minimal with CASP-aware OS scheduling in multicore systems. We modified the scheduler of a Linux OS (version 2.6.25.9) to consider the unavailability of cores undergoing online diagnostics. The scheduler migrates and schedules tasks intelligently around online diagnostics to minimize performance impact. Experimental results on a dual quad-core Xeon system show that the performance impact of CASP on noninteractive applications is negligible when the system is not fully utilized: on average, overhead of $< 1\%$ latency is obtained for the PARSEC benchmark suite. In addition to evaluation for noninteractive applications, we evaluated performance for interactive and soft real-time applications.

Figure 4c shows response times as a cumulative function for a common interactive application, the Firefox web browser. With hardware-only CASP, approximately 15% of all user actions suffer from delays of more than 500 ms, which is classified as unacceptable for humans in human-computer interaction literature. On the other hand, with CASP-aware OS scheduling, all response times are less than 200 ms,

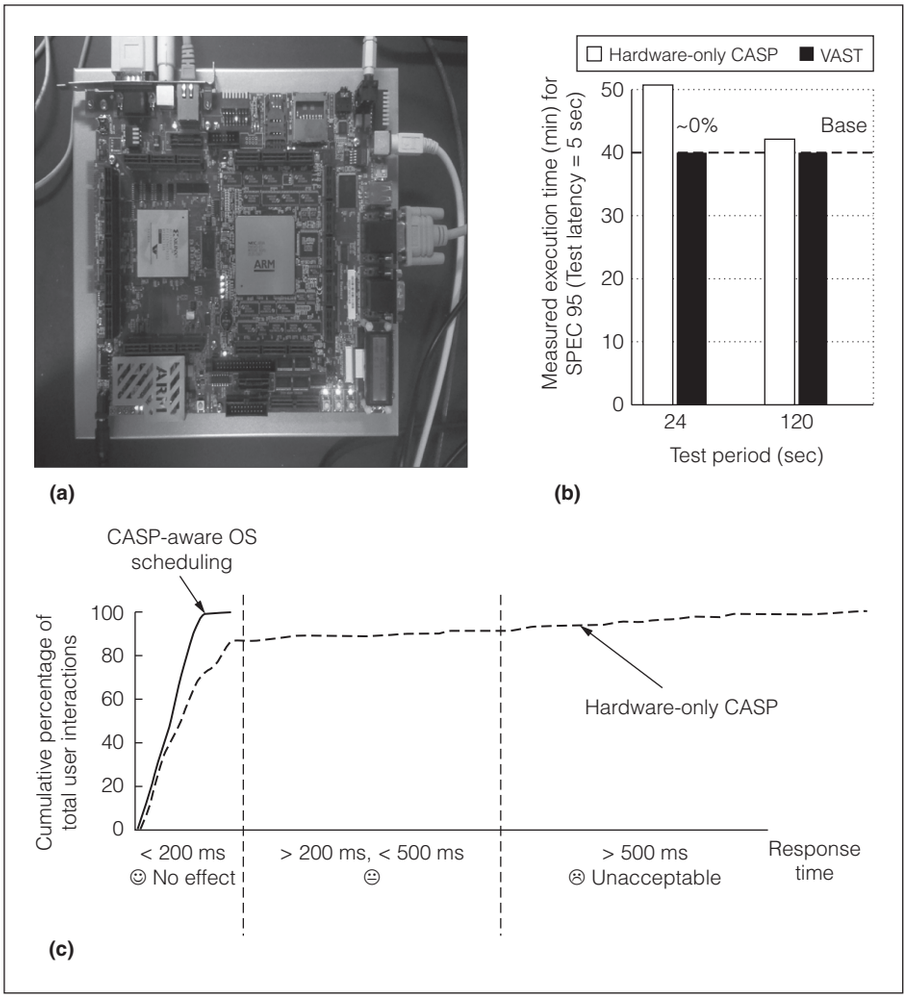

Figure 4. The VAST platform (Source: Inoue et al.[12] Courtesy NEC Corp.) (a), evaluation results of VAST for SPEC 95 benchmark suite (b), and Firefox response times comparing CASP-aware OS scheduling and hardware-only CASP (c).

which is below the perceptible threshold for humans. This figure clearly shows the superiority of CASP-aware OS scheduling compared to hardware-only CASP.

Although our description of CASP is limited to processor cores, special CASP-like techniques must be developed for the so-called "uncore" part (or the non-processor core) of a SoC. CASP can be further extended to enable *adaptive online diagnostics* by adjusting various parameters (summarized in Table 4) based on system characteristics such as targeted reliability mechanisms, system workload, and operating conditions. Figure 5 shows an example of adaptive online diagnostics using CASP. In this example, proactive CASP runs periodically for circuit failure prediction, and reactive CASP runs when system checkers or sensors detect a system anomaly (e.g., if the system reaches a critical temperature, or residue or parity checkers detect errors).

Optimized self-healing

Self-healing is an essential component of robust system design. The term "self-healing" is generally used in robust systems literature to refer to systems with correction and containment of faults that enable them to recover from unexpected changes or failures with no or very little impact on the system level (e.g., IBM's autonomic computing initiative; see http://www-03.ibm.com/servers/autonomic). The use of a particular self-healing procedure depends on the type of detected or predicted failure. For systems with error detection, various recovery and repair techniques for transient errors and permanent faults exist in commercial implementations (as an example, see the work of Meaney et al.[13]).

We focus on unique opportunities that exist in designing systems with optimized self-healing in the presence of transistor aging. Unlike traditional error recovery, optimized self-healing for aging progressively tunes a set of tuning parameters—mainly supply voltage and clock frequency—to compensate for aging and to prevent any delay-fault-induced errors due to aging. Policies for such an approach attempt to maximize two elements:

- An energy efficiency metric, *computational power efficiency* (CPE), which is defined as the total number of clock cycles achieved over a system's lifetime divided by the total energy.
- Overall *system lifetime,* which is defined as the point in time at which the peak performance demand can no longer be satisfied (for given constraints on tuning parameters).

Table 4. Parameters for CASP online diagnostics.

Parameter	Definition	Example
Invocation mechanism	The mechanism that triggers the start of CASP.	Proactive: test controller predetermines the times when CASP needs to run. Reactive: CASP runs in response to certain system events and errors.
Pattern set	The set of patterns that will be applied during online diagnostics. It is one factor that determines the test latency.	Test type: path delay patterns or transition fault patterns. Location: patterns that specifically target the ALU, the floating-point unit, and so on.
Test latency	How long the test runs. It depends on the pattern set, the length of the longest scan chain, and the test compression technique.	For the OpenSparc T1, it takes 300 ms to run all of the stuck-at, transition, and True-Time patterns, when using 10× test compression.
Test period	How often the test runs for periodic online diagnostics.	For circuit failure prediction for ELFs, online diagnostics might run with a period in the order of tens of seconds; for aging, the test period could be longer—say, once a day.

This is a challenging problem because the choice of tuning parameters made at any one point in time affects future aging, power, and performance. Supply voltage can be increased to reduce delay; however, dynamic and leakage power will increase, and aging will worsen. Reducing clock frequency can prevent errors and also reduce dynamic power, but system performance degrades.

Two representative policies are important in this context: *progressive-greedy* (PG) and *progressive-real-time-aging-assisted* (PRTA), which are illustrated in Figures 6a and 6b. Both policies follow the flowchart in Figure 2. For PG, after a time step, the system adjusts its self-healing parameters (e.g., supply voltage and clock frequency) assuming worst-case aging during the preceding and succeeding time steps, and the

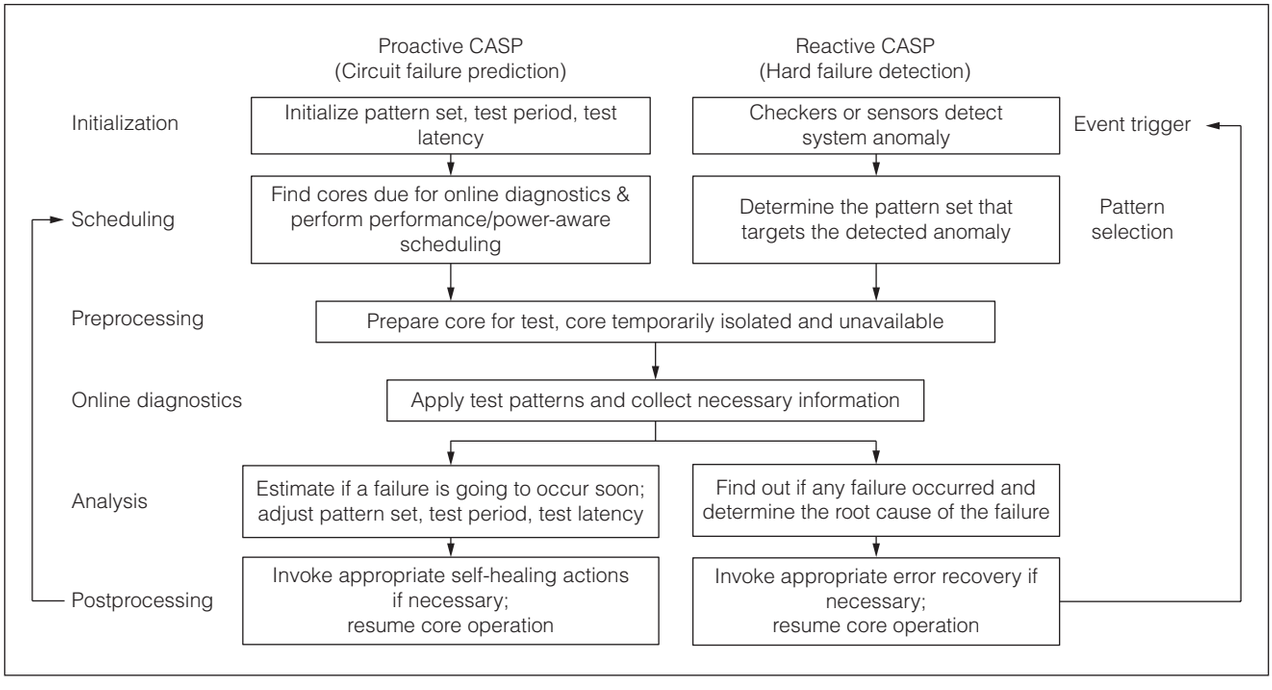

Figure 5. Adaptive CASP online diagnostics.

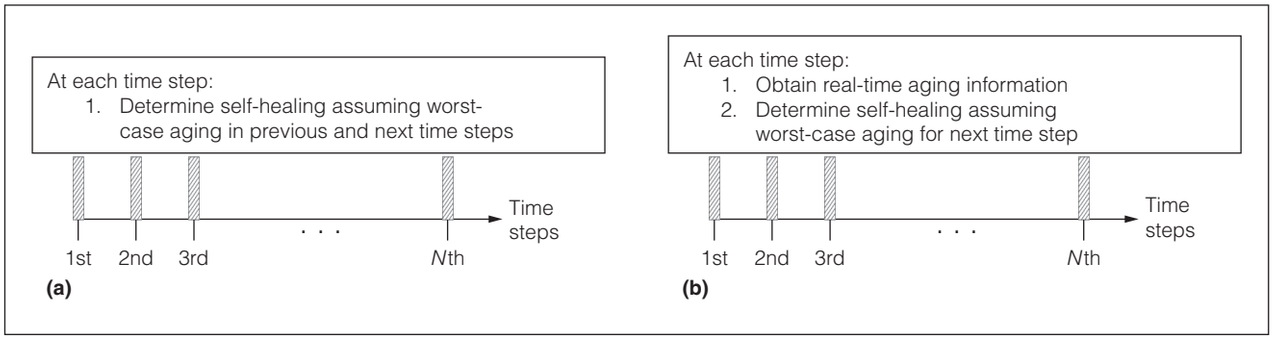

Figure 6. Progressive-greedy (PG) self-healing scheme (a), and progressive-real-time-aging-assisted (PRTA) self-healing scheme (b).

loop continues. PG achieves its benefits by performing self-healing progressively over a system's lifetime rather than only once in the beginning. In contrast, PRTA is assisted by real-time aging information using circuit failure prediction—for example, delay shifts detected using CASP online diagnostics. Hence, unlike PG, PRTA does not assume worst-case aging during the previous time step. Instead, it uses an estimate of the actual amount of aging and adjusts system self-healing (assuming worst-case aging for the succeeding time step). The use of optimization algorithms implies that one or more tuning knobs may or may not be adjusted at the end of any time step.

Of course, PRTA is beneficial when a system is not stressed to the worst-case level of aging all the time, which is realistic for many system usage patterns, especially with the support of dynamic voltage and frequency scaling (DVFS) that is available in most systems today.

Extensive simulations using aging models, calibrated with 45-nm silicon data from aging experiments,[6] can help derive a set of simple system design guidelines for PRTA:

- The choice of a specific self-healing technique for transistor aging strongly depends on workload characteristics. For a system that is active 20% of the time, with a performance requirement of 2.5 GHz and an average stress level that corresponds to 20% of worst-case aging, PRTA recovers 88% of CPE degradation induced by one-time worst-case guardbanding.
- The use of specific self-tuning knobs (frequency vs. supply voltage) depends on the specific application requirements (that is, if there is a specific minimum frequency requirement at all times).
- Time steps of one day are sufficient for both PG- and PRTA-based policies.
- For PRTA-based self-healing, attention must be paid to the resolution and the cost of supporting techniques for aging estimation. For example, for a 1.5-GHz system, a target resolution of 15 ps and target power costs of less than 1% are desired.
- For maximized benefits of self-tuning, 10 to 15 voltage levels might be required with a resolution of 12.5 mV (already supported in today's systems).

THE GREATEST CHALLENGE in designing robust systems is to minimize the costs of error detection. Most existing error detection techniques suffer from high power and performance costs, and/or additional design complexity. Circuit failure prediction, together with CASP online diagnostics, enable the design of robust systems that can effectively overcome reliability challenges associated with gate-oxide early-life failures and circuit aging, because of the gradual nature of degradation associated with these mechanisms. The key attractive feature of such an approach is its significantly reduced power cost compared to traditional error detection. It also opens up new research opportunities across multiple abstraction layers (circuit, architecture, virtualization software and operating systems, and applications) for designing optimized robust systems with respect to reliability requirements while balancing power, performance, area, and design complexity constraints. Such global optimization is essential for robust systems of the future. ■

Acknowledgments

Our research is supported in part by the Focus Center Research Program (FCRP) Gigascale Systems

Research Center (GSRC) and Center for Circuit and System Solutions (C2S2), the National Science Foundation (NSF), the Semiconductor Research Corporation (SRC), Bosch, IBM, Infineon, Intel, RobustChip, and Texas Instruments.

References

1. B. Schroeder and G. Gibson, "Understanding Failures in Petascale Computers," *J. Physics: Conference Series* (SciDAC 07), IOP Publishing, 2007, vol. 78, article 12022.
2. S. Shazli et al., "A Field Analysis of System-Level Effects of Soft Errors Occurring in Microprocessors Used in Information Systems," *Proc. Int'l Test Conf.* (ITC 08), IEEE CS Press, 2008, pp. 1-10.
3. S. Borkar, "Designing Reliable Systems from Unreliable Components," *IEEE Micro,* vol. 25, no. 6, 2005, pp. 10-16.
4. S. Mitra et al., "Robust System Design with Built-In Soft Error Resilience," *Computer,* vol. 38, no. 2, 2005, pp. 43-52.
5. J.M. Carulli Jr. and T.J. Anderson, "The Impact of Multiple Failure Modes on Estimating Product Field Reliability," *IEEE Design & Test,* vol. 23, no. 2, 2006, pp. 118-126.
6. R. Zheng et al., "Circuit Aging Prediction for Low Power Operation," *Proc. IEEE Custom Integrated Circuits Conf.* (CICC 09), IEEE Press, 2009.
7. S. Mitra and E.J. McCluskey, "Which Concurrent Error Detection Schemes to Choose?" *Proc. Int'l Test Conf.* (ITC 00), IEEE CS Press, 2000, pp. 985-994.
8. M. Agarwal et al., "Circuit Failure Prediction and Its Application to Transistor Aging," *Proc. 25th IEEE VLSI Test Symp.* (VTS 07), IEEE CS Press, 2009, pp. 277-286.
9. H. Baba and S. Mitra, "Testing for Transistor Aging," *Proc. 27th IEEE VLSI Test Symp.* (VTS 09), IEEE CS Press, 2009, pp. 215-220.
10. T.W. Chen et al., "Experimental Study of Gate-Oxide Early Life Failures," *Proc. Int'l Reliability Physics Symp.* (IRPS 09), IEEE Press, 2009, pp. 650-658.
11. Y. Li, S. Makar, and S. Mitra, "CASP: Concurrent Autonomous Chip Self-Test Using Stored Test Patterns," *Proc. IEEE Design, Automation, and Test in Europe* (DATE 08), IEEE CS Press, 2008, pp. 885-890.
12. H. Inoue, Y. Li, and S. Mitra, "VAST: Virtualization-Assisted Concurrent Autonomous Self-Test," *Proc. Int'l Test Conf.* (ITC 08), IEEE CS Press, 2008, pp. 1-10.
13. P. Meaney et al., "IBM z990 Soft Error Detection and Recovery," *IEEE Trans. Device and Materials Reliability,* vol. 5, no. 3, 2005, pp. 419-427.

Yanjing Li is a doctoral candidate in electrical engineering at Stanford University. Her research interests include architectural and system-level techniques for robust-system design. She has an MS in mathematical sciences from Carnegie Mellon University.

Young Moon Kim is a doctoral candidate in electrical engineering at Stanford University. His research interests include design methodology for robust systems. He has an MS in electrical engineering from Purdue University.

Evelyn Mintarno is a doctoral candidate in electrical engineering at Stanford University. Her research interests include design for reliability. She has an MS in electrical engineering from Stanford University.

Donald S. Gardner is a principal engineer at Intel Research and is also a visiting scientist in the Electrical Engineering Department at Stanford University. His research interests include future microprocessor technology, magnetic materials for inductors, silicon-based optoelectronic devices, nanostructure design and devices, and process integration. He has a PhD in electrical engineering from Stanford University.

Subhasish Mitra is an assistant professor in the Departments of Electrical Engineering and Computer Science at Stanford University, where he leads the Stanford Robust Systems Group. His research interests include robust-system design; VLSI design, CAD, validation, and test; and design for emerging nanotechnologies. He has a PhD in electrical engineering from Stanford University.